Streamlining Loss Run Data Extraction: An Automated Solution Using AI and Cloud Services

Katie Colaosno, Shriya Purohit, Carmen Yu

April 11, 2025

1 Introduction

Commercial insurance conducts risk assessments through the availability of accurate historical information. One of these vital documents is called a loss run report, which provides a snapshot of an insured entity's claims history, detailing claim frequency, and loss severity. For underwriters, brokers, and insured parties, these documents serve as both a record of risk exposure and a tool to forecast future liabilities. As the insurance industry embraces digital transformation, automating the extraction and interpretation of loss runs has become increasingly important - particularly given the unstructured nature of these documents and the variability of their formats across carriers.

The purpose of this project was to automate the extraction, structuring, and analysis of loss run documents to support the underwriting and risk assessment for Crum and Forster. Traditional methods for processing loss run reports are manual, time-consuming, and prone to errors due to their unstructured formats. Loss run reports can vary widely in appearance and format. For example, they might include a scanned copy of a handwritten summary, a brief overview of a previous claims with limited detail, embedded charts, or be delivered in formats like .csv files instead of PDFs. Some may consist solely of insured policy information without any claims data. These documents tend to be stored across multiple sources creating challenges in retrieval and analysis. Our solution leverages artificial intelligence, APIs, and cloud-based services to streamline this process, improving efficiency and accuracy.

While the automation process offers significant benefits, several key challenges must be addressed to fully optimize the processing of loss run reports. The primary difficulty encountered in this project included the fragmentation of data in multiple company specific databases such as ImageRight, Workbench, and uDoc. Additionally, the unstructured nature of the loss run reports made automated data extraction difficult. Manual processing bottlenecks further slowed underwriting decisions and cost considerations, particularly related to the use of AWS Textract OCR processing, which added another layer of complexity. These challenges underscore the importance of developing a robust and scalable solution that can seamlessly integrate and process data from disparate sources while minimizing errors and delays.

To address these challenges, we developed an automated pipeline that retrieves loss run

data from multiple databases, processes the unstructured information, and structures it into a consistent format based on a predefined output schema. The pipeline integrates multiple tools and technologies to achieve a structured format.

The end product of this project resulted in an Flask REST API deployed on Posit Connect for easy user access to the AI integrated tool that takes the unstructured loss run documents as inputs and outputs structured JSON files. However, the final product was completed in several phases. The first phase was a hackathon in which individual teams tackled creating a data cleaning and API-integrated pipeline; guided by industry experts. In the second phase, each team was provided with a line of business (LOB) to optimize their solution and provided accuracy measures using uDoc and Workbench as ground truths. In the final phase, each team was assigned a task that was dependent on the final output of another team. These cascading tasks included harmonization of the code from all teams (i.e., combining the best aspects of each teams' code), refining accuracy measures, deploying and testing a Flask API, creating a demo video of how the whole pipeline works and creating a business presentation for Crum and Forster management.

2 Fast-Tracking Automation: The Hackathon Phase

2.1 Loss Run Processing Workflow

To accelerate automation efforts during the hackathon, we developed a modular pipeline to process insurance loss run documents. This workflow automates document retrieval, extraction, and structuring for downstream analysis. A high-level overview of this workflow is provided in Figure 1, which illustrates the document pipeline from ingestion to structured output. The core functionality of this solution was implemented through various classes, each handling a specific task in the workflow.

Data extraction begins by gathering metadata for the loss run documents through the `ImageRightLossRuns` class. This class automates the retrieval and processing of loss run documents for product numbers by interacting with the ImageRight API. It reads product numbers from a .csv file, makes APIs calls to fetch metadata, extracts loss run IDs, and retrieves PDFs. These PDFs are then processed and concatenated into single files for each product number, saved with timestamped filenames in a specified directory. The class includes error handling for cases where no loss runs or PDFs are found, and saves these product numbers in a separate .csv file for easy reference. The `ImageRightLossRuns` class allows for quick retrieval of many loss run documents.

Once the PDFs are retrieved, the information is extracted through the `AWSTextract` class, which automates the extraction of the text and data from the PDF files. The class processes PDF files from a designated folder, encodes them in Base64, and sends them to the AWS Textract API for parsing. AWS Textract parses documents using optical character recognitions (OCR) and machine learning. The resulting data is saves as JSON files, timestamped and labeled by product number. The class includes error handling for server-side issues and failed requests, ensuring large batches of documents can be processed reliably.

The extracted text is then processed through the `ChatGPTProcessor` class, which utilizes OpenAI's ChatGPT to format the text into a standardized nested JSON format for easy analysis.

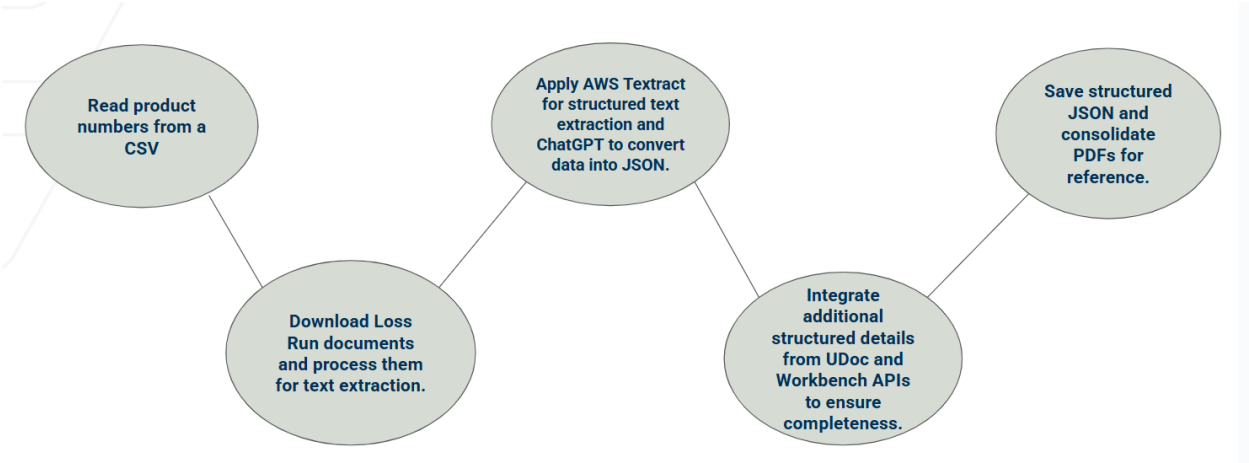

Figure 1: A simplified version of the process workflow for the AI automated process of structuring unstructured loss runs.

The class begins by preparing the data for analysis through helper functions like `loadJSON` and `saveJSON` for reading and writing JSON files. Another helper function extracts blocks of text and saving it in a format suitable for API calls.

ChatGPT was given a carefully designed prompt with domain-specific instructions and a predefined output schema to extract key fields such as carrier, policy number, insured name, and detailed claim information. Specifically, claims are grouped under a single parent JSON object representing the policyholder, ensuring that related claims are consolidated together. Because our version of ChatGPT was stateless, we introduced a short-term memory mechanism in the prompt, allowing the model to reference the last three pages of content. This helped maintain consistency across multi-page documents and ensured that related claims were properly nested under the correct policyholder, even when the information was spread across different sections of the document. The `processText` and `processJSONPages` methods handled retries, API cost calculations, and aggregation of output into a structured, nested JSON format that consolidates all claims under the appropriate policyholder. A final `cleanData` function ensures completeness and consistency across documents. These structured JSON files were saved and organized in designated folders for downstream analysis.

To supplement missing or incomplete information, additional structured data was pulled using the `uDocLossRuns` and `WorkbenchLossRuns` classes. These tools automated API calls using product numbers from a .csv file to fetch metadata and documents from internal systems. If PDFs are found, they are downloaded, concatenated, and saved with timestamped filenames for easy reference. These systems serve as internal sources of ground truth, though coverage varies across LOBs, with some documents available in only one of the two databases. A summary of our final outputs is provided below.

1. Structured JSON files: contains well-organized loss run data

2. Merged PDFs: consolidated loss run reports for each product number

3. Logs of skipped products: for missing loss run documents that could not be extracted

4. Supplementary structured data: retrieved from uDoc and Workbench APIS

2.2 Challenges

During the hackathon, we encountered several challenges while developing the modular pipeline. The first challenge arose when attempting to retrieve PDFs from ImageRight, as many were missing. We discovered that non-PDFs were included in the API results, which AWS Textract could not process. To address this, we implemented a filter to exclude non-PDF files before processing. The second challenge was managing the cost of AWS Textract API calls. To minimize expenses, we decided to limit testing to a subset of product numbers. Third, we initially assumed that rate limits for ChatGPT were causing numerous stats call response errors. However, we later realized that the issue stemmed from oversized prompts, and we implemented prompt chunking to resolve this. Lastly. we faced difficulties with highly nested and inconsistent JSON structures in uDoc and Workbench. To overcome this, we established a standard schema, with input from industry experts, to ensure consistent and structured data across all sources.

3 Tailoring the Solution: The Optimization Phase

The optimization phase began with each team being assigned a specific LOB. Our team focused on workers' compensation, which presented unique challenges due to its complexity and the way it is embedded within broader policy claims. For example, if an employee is injured while driving a company-owned vehicle, the resulting claim may include both auto-related and workers' compensation components. In such cases, details about the injury, like a fractured collarbone, must be identified as part of the workers' compensation claim, even though they're not filed separately. This required our system to analyze entire policy claims and extract relevant information regarding workers' compensation.

3.1 Understanding Workers' Compensation

Before diving into our optimized solution, it is important to understand the dynamics of how workers' compensation is represented in loss run documents. In general, workers' compensation provides cash benefits and/or medical care for employees who are injured on the job [1]. Loss run documentation will include the cash benefits and medical care payouts that a policy holder has claimed. These are usually separated into two different sections - medical and indemnity. If an employee is unable to return to work due to their injuries, the policy holder will pay out the lost wages, otherwise known as indemnity.

Indemnity claims generally appear in three forms:

1. **Indemnity payments**: payments for lost

wages

2. **Indemnity reserves**: estimated future costs related to wage compensation

3. **Indemnity recovery**: payments received from third parties or other sources to offset lost wage payments

Medical and incident information within a loss run document is usually well documented. It typically includes information on the description of the injury and where the injury occurred on the on the body. Medical payment information has a similar structure to indemnity payments in a loss run document, as well.

It is typically categorized into three parts:

1. **Medical payments**: amount paid for treatments

2. **Medical reserves**: estimated future medical payments

3. **Medical recovery**: amounts received from third parties or other sources to offset medical costs

The last category that may be included in a loss run document for workers' compensation is related to employer liability. Workers' compensation typically operates under a no-fault assumption, meaning neither the employee nor employer is considered at fault, and compensation isn't reduced due to perceived negligence [1]. Of course, there are exclusions to this rule if the activity involved alcohol or drugs [1]. Now, an employee may decide to sue the company if he or she believes that the company was negligent. In these cases, the loss run documentation may include information on expense payments associated with any payments for losing a workers' compensation case.

3.2 Optimizing Our Approach

Prompt Engineering To optimize our approach for workers' compensation loss run documents, we refined the prompt in the `ChatGPTProcessor` class to focus on relevant fields. These included accident descriptions, medical information, injury details, employee identifiers, indemnity and medical payments.

Data Summarization After processing, we passed the output through the `cleanData` function. We added functionality to generate a summary JSON object for each policy, giving users a quick overview of how many claims were filed over a given time period.

Performance Metrics To ensure accuracy, we implemented several consistency checks. Since language models like ChatGPT can generate plausible but incorrect information (also known as hallucinations), it was important to validate the output.

We created a `calculateMAE` function to compute the Mean Absolute Error (MAE) by comparing ChatGPT-extracted values to ground truth data from uDoc and Workbench. We focused on key fields like paid losses, reserves, recoveries, deductibles and total incurred losses. The function calculates the absolute differences between the extracted and ground truth values - and tracks missing polices and claims.

The final check was through the `internalConsistency` function which assessed structural accuracy. It checks for discrepancies between the reported claim counts, formatting and calculations of incurred and net incurred losses from Workbench and uDoc and the ChatGPT-extracted values.

The results are summarized in table ... showing average MAE and internal consistency scores across 80 documents. As expected for an early

prototype, the error rates were initially high, but they establish a baseline for ongoing refinement. Lower scores will indicate better performance as the system improves over time.

Field	Average MAE
Paid Loss	78,180
Reserves	22,733
Deductible	242
Total Incurred	111,722
Missing Policy Count	14
Missing Claim Count	18

Field	Average Internal Consistency
Row Count	191
Claim Count	-19
Total Incurred	1,688
Net Incurred	13,818

4 Final Touches: The Deployment Phase

The final phase brought all teams together to deliver a final polished product. The first team began by compiling everyone's optimized code into one cohesive, harmonized solution. The goal was to finalize a prompt and method that captured the optimization each team had done for specific carriers and LOBs. Once harmonized, the next team focused on refining a consistency function to assess for internal coherence in ChatGPT's responses; similar to the internal consistency checks implemented during the "Tailoring the Solution" phase. Our team was responsible for building the Flask API that would deploy the final harmonized code, while the last two teams would perform research on edge cases and additional accuracy metrics based on existing ground truths. After our deployment, the last team prepared a business presentation for Crum and Forster management.

4.1 Flask API

Our Flask API takes in a Base64-encoded string representing the loss run document. This format helps preserve page breaks and make it easier to split the document into individual pages. The API also accepts additional parameters to fine-tune ChatGPT's processing if necessary. Once received, the Base64 string is processed through additional APIs to Amazon Textract and ChatGPT, producing a JSON output. The full process and prompted used are based on the harmonization functions done by the first team. This output is then reshaped by us into the final schema preferred by Crum and Forster, as announced at the end of the hackathon. The final result is a structured JSON file containing extracted information such as previous claims and policies; streamlining what was previously a manual and time-intensive data entry process. In addition, the internal consistency of the response was checked using the consistency function built by another team before returning the API results. The API was then locally tested and deployed through Posit Connect to be available for use for Crum and Foster.

5 Conclusion

Commercial insurance heavily relies on loss run reports to be able to conduct thorough risk assessments. Loss run reports generally provide detailed views of loss incidents. However, these reports tend to be scattered amongst multiple databases and in disorganized formats. Under-

writers and brokers must manually aggregate this data in order to get a complete picture of all current and historical policy claims for an insured individual. This can lead to human prone errors that greatly reduce the profitability of an insurance company.

Crum and Forster wanted to create an automated approach to reduce the manual aspect of this industry and to enable quick and easy analysis of loss run documentation. Our approach provided automation of several key steps in gathering loss run documents. The final product of this approach resulted in a deployed API for easy user access that takes in unstructured loss run documents as inputs and outputs structured JSON files. Accuracy and internal consistency measures are included to indicate how well the automated pipeline is selecting and organizing the relevant information. Users will be able to get a clear view of how well the model is performing every time they make a request to the API.

References

[1] New York State. *What is Workers' Compensation?* URL: https://www.wcb.ny.gov/content/main/Workers/what-is-workers-compensation.jsp#.